Introducción a la seguridad en SOs

Sistemes Operatius II

Oliver Díaz

Grau d'Enginyeria Informàtica

Objetivos de la sesión

- Entender los peligros existentes de seguridad de los PCs
- Conocer (algunas) vulnerabilidades que afectan a los SOs
- Familiarizarse con los mecanismos de protección del SO

¿Reconocéis esta aula?



- Carla guarda su informe de la P1 en un fichero durante el laboratorio del jueves.
- Juan utiliza la misma máquina durante el grupo del viernes.

¿Qué debería impedir el SO?

¿Leerlos? ¿Cambiarlos? ¿Dejarla sin espacio?

Máquinas compartidas



- Carla guarda su informe de la P1 en un fichero durante el laboratorio del jueves.
- Juan utiliza la misma máquina durante el grupo del viernes.

Objetivo	En nuestro ejemplo
Confidencialidad	Juan no lee informe de Carla
Integridad	Juan no modifica el informe de Carla
Disponibilidad	Carla puede acceder cuando los necesita

Máquinas compartidas



- Carla guarda su informe de la P1 en un fichero durante el laboratorio del jueves.
- Juan utiliza la misma máquina durante el grupo del viernes.

El SO tiene mecanismos para contribuir a estos objetivos, pero no los garantiza por sí solo.

Ataque UAB (2021)

Un ataque informático con 'ransomware' deja la Universitat Autònoma de Barcelona sin internet

Basta con que un trabajador caiga en la trampa para que todo el sistema informático de una empresa caiga. El anzuelo acostumbra a ser un **correo electrónico falso,** aparentemente relacionado con alguna gestión habitual en la rutina de una organización. Un clic de un empleado confiado y el virus infecta la plataforma cifrando y bloqueando su acceso. Así funciona el **virus ransomware**, el ataque informático que más se repite contra las empresas. Hay dos maneras de sortearlo: pagar el **rescate económico** que solicita el ciberdelincuente, o bien restaurar el sistema por completo, un trabajo titánico, en ocasiones hasta imposible, según relata Jordi Ferrer, vocal de la comisión digital del Colegio de la Abogacía de Barcelona (ICAB).

Ataque UOC (2022)

Ataque informático contra la UOC que paraliza su campus virtual

Los supuestos atacantes envían una nota para que la universidad se ponga en contacto con ellos

El ataque se consideró, a lo largo de la tarde de este domingo, que podía ser un problema técnico. Pero ha resultado ser **un ataque de tipo ransomware, que restringe el acceso a determinadas partes del sistema.** Y en ocasiones se llega a pedir un rescate para levantar la restricción.

https://metropoliabierta.elespanol.com/el-pulso-de-la-ciudad/20220102/ataque-informatico-contra-la-uoc-que-paraliza-su-campus-virtual/639436109_0.html

Ataque Hospital Clínic (2023)

Ciberataque al hospital Clínic de Barcelona



https://www.clinicbarcelona.org/prensa/ultima-hora/ciberataque-al-hospital-clinic-de-Barcelona

5 de julio del 2023. 17:41h

Comunicado ciberataque

Confirmamos que RansomHouse hizo este martes una nueva publicación de datos. Esta publicación incluye la totalidad de los datos robados en el ciberataque del pasado 5 de marzo (4,5TB). La publicación incluye varios datos de carácter personal de pacientes, profesionales y entidades colaboradoras y proveedores.

El Hospital Clínic de Barcelona, como Responsable del Tratamiento, de acuerdo con lo que establece el Reglamento (UE) 2016/679 de 27 de abril relativo a la protección de las personas físicas en relación con el tratamiento de datos personales, procederá a actualizar en su web la información de la que dispone, en relación con el compromiso de la confidencialidad de los datos. Ésta es la vía oficial para comunicar a los afectados la publicación de datos personales

Ataque UB (09/2026)

UB >

La Universidad de Barcelona investiga el alcance de un ciberataque y garantiza el proceso de matrícula

La institución asegura que mantiene la actividad con normalidad, salvo algunos servicios y páginas web

https://elpais.com/espana/catalunya/2026-08-31/la-universidad-de-barcelona-investiga-el-alcance-de-un-ciberataque-y-garantiza-el-proceso-de-matricula.html

La institución ha activado los protocolos de seguridad y ha adoptado medidas de contención, monitorización y refuerzo de sus sistemas. Entre las precauciones adicionales figura la renovación de las credenciales de acceso y la interrupción puntual de algunos servicios y páginas web. Estas restricciones, adoptadas como medida preventiva, pueden mantenerse durante los próximos días mientras continúa la investigación. Algunos profesionales explican a EL PAÍS que en los últimos días habían tenido dificultades para acceder a algunas plataformas, aunque se desconoce si estas dificultades estaban relacionadas con el incidente. La universidad, en todo caso, asegura que hasta el momento "no se ha detectado ninguna incidencia" que afecte a sus datos, aunque los técnicos continúan trabajando para determinar la magnitud del ataque y "la información potencialmente" afectada.

WannaCry

- Gusano ransomware (mayo de 2017)
- Infectó 200,000+ ordenadores
- 150+ países
- FedEx, Honda, Nissan, NHS, …



- **Vulnerabilidad “EternalBlue”** para MS Windows (desarrollada por la NSA)

Amenaza, vulnerabilidad y riesgo

- **Amenaza**: alguien (o algo…IA) intenta alterar los resultados
- **Vulnerabilidad**: un permiso permite escribir a cualquiera
- **Riesgo**: posibilidad e impacto de que esa alteración ocurra

¿Un error de programación siempre es explotable?

Vulnerabilidad



“Debilidad o fallo en un sistema informático (seguridad, diseño, implementación, controles internos) que podría ser explotada para comprometer la confidencialidad, integridad o disponibilidad del sistema”

¿Las vulnerabilidades son evitables?

- ¿Cuantas líneas de código tiene un SO?
  - El *kernel* de Linux 6.5-rc5 (2023) contiene **~36 millones de líneas de código**
- ¿Cuántos errores cometemos al programar?
  - "An exceptionally good software development process can keep defects down to as low as **1 defect per 10,000 lines of code**"

***NASA** Study on Flight Software Complexity*

¿Las vulnerabilidades son evitables?

- Errores de código → Vulnerabilidades potenciales
- ¿Ejemplo de vulnerabilidades?:
  - *Buffer overflow*
  - Uso después del `free()`
  - *Null pointer*

Common Vulnerabilities and Exposures (CVE)

- El propósito de CVE es identificar, definir y catalogar **vulnerabilidades de ciberseguridad** divulgadas públicamente de forma estandarizada.
- Facilita a los profesionales, investigadores y organizaciones de seguridad compartir información sobre problemas de seguridad.



https://www.cve.org/

Las vulnerabilidades son inevitables

- "En enero de 2021, *Qualys* descubrió un ***heap overflow*** en `sudo` que permitía a los usuarios ejecutar programas con los **privilegios de otros usuarios**. Este bug fue introducido en 2011 (CVE-2021-3156) y afectó a Linux, Mac OS y BSD."

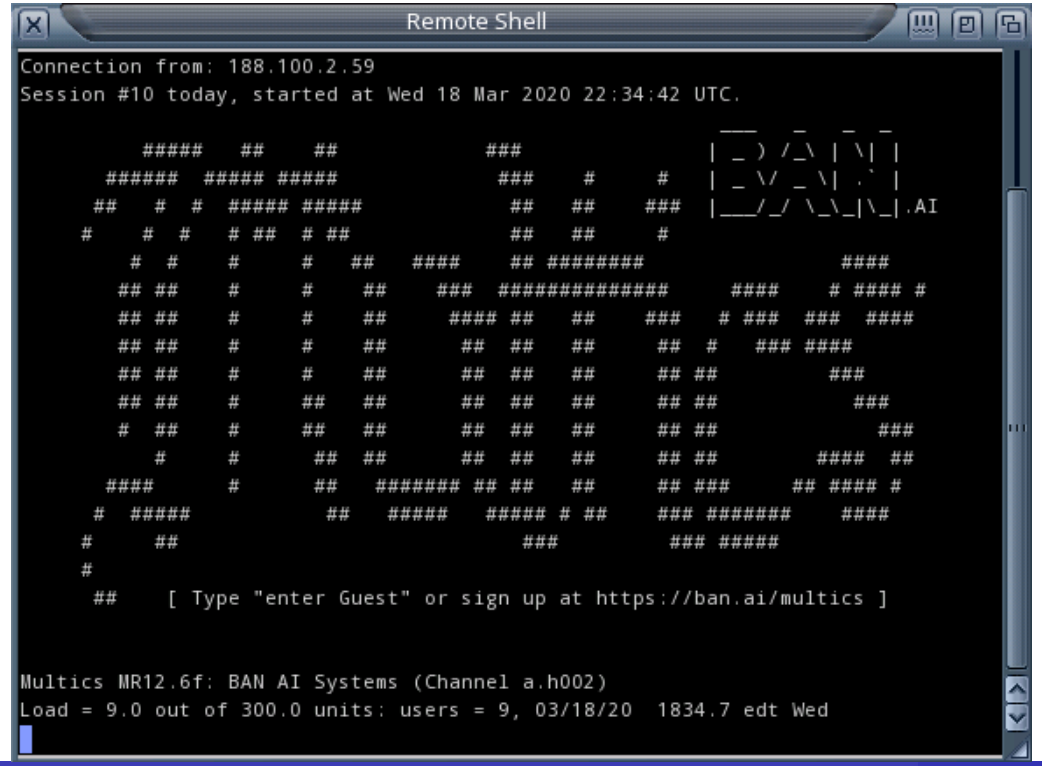
```
greys@mcfly ~ sudo -V
Sudo version 1.9.5p2
Sudoers policy plugin version 1.9.5p2
Sudoers file grammar version 48
Sudoers I/O plugin version 1.9.5p2
Sudoers audit plugin version 1.9.5p2
```

Las vulnerabilidades son inevitables

- Lenguajes C y C++
- **Python** esta escrito en C y **también sufre vulnerabilidades**
- **CVE-2016-5636**: El desbordamiento de enteros en la función `get_data` en `zipimport-c` de `Cpython` permite a atacantes remotos tener un impacto no especificado a través de un valor de tamaño de datos negativo, lo que desencadena un *heap overflow*.

Un poco de historia: MULTICS

- Multiplexed Information and Computing Service (MULTICS)
- SO (1965-2000) de MIT, Bell Labs, GE **pionero en seguridad informática**
  - Recursos compartidos
  - Protección de memoria
  - Acceso controlado por permisos
  - Jerarquías de directorios



Principios de sistemas seguros

- Defensa multinivel
- Principio del menor privilegio (*least privilege*)
- Separación de privilegios
- …

Principios de sistemas seguros

- **Defensa multinivel**
- Principio del menor privilegio (*least privilege*)
- Separación de privilegios

Defensa multinivel

- Los sistemas deberían proveer de protección a múltiples niveles

Capa	Ejemplo de protección
Aplicación	Comprobar la longitud de entrada
Proceso	Limitar sus permisos y recursos
SO y hardware	Aislar memoria y controlar ejecución
Operación del sistema	Actualizar y preparar recuperación

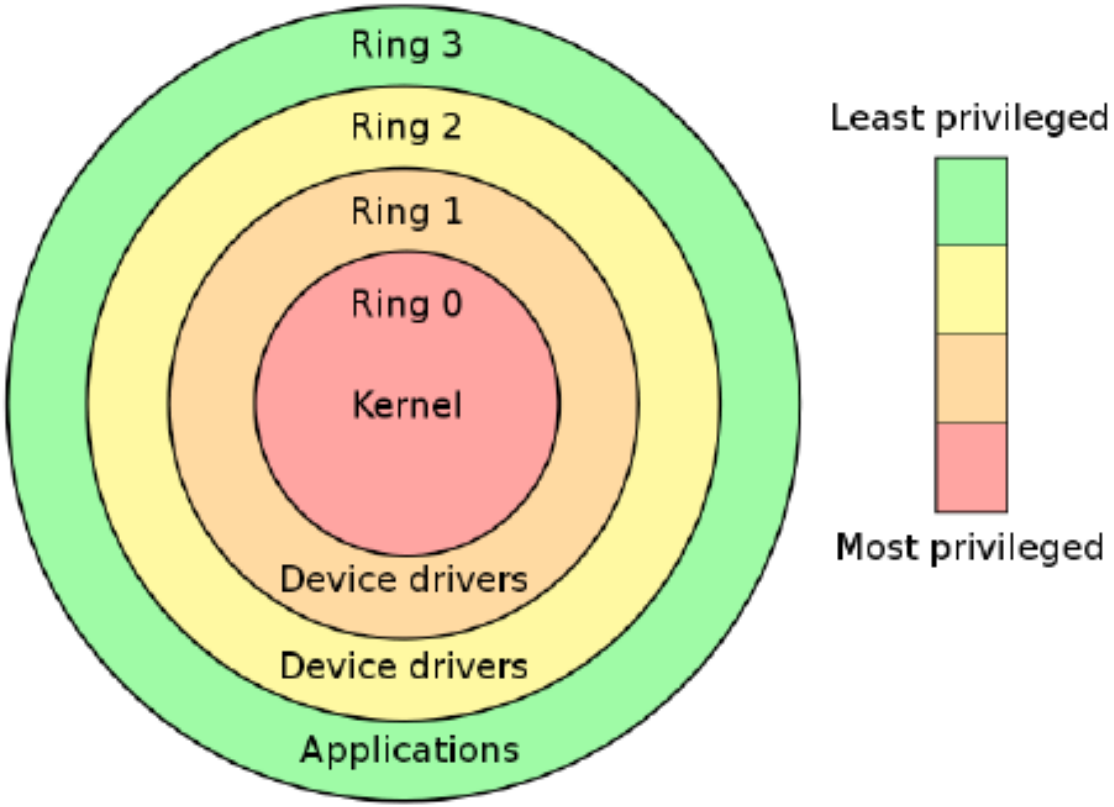
Defensa multinivel

- Los sistemas deberían proveer de protección a múltiples niveles
- P. ej. Vulnerabilidad en el **intérprete de Javascript del Chrome**.
  - Chrome (**proceso**) debería prevenir el acceso a otras pestañas
  - **SO** debe prevenir el acceso a otros procesos (p.ej. Gestor de contraseñas)
  - **Hardware** debería prevenir instalación de malware en el firmware de los dispositivos
  - La **red** debería prevenir la propagación del malware a otros ordenadores



Defensa multinivel

- CPUs modernas soportan 4 anillos de protección, pero usan 2
- Intel x86
  - Anillo 0 tiene el *kernel*
  - Anillo 3 contiene código apps



https://en.wikipedia.org/wiki/Protection_ring

Defensa multinivel

1. El proceso solicita abrir el fichero (`open`)
2. La llamada al sistema entra en el *kernel*
3. El *kernel* comprueba credenciales y reglas
4. Devuelve un descriptor (`fd`) o un error

Modo usuario ↔ **Modo *kernel***

establecen una frontera de protección

Principios de sistemas seguros

- Defensa multinivel
- **Principio del menor privilegio (*least privilege*)**
- Separación de privilegios

Principio del menor privilegio

- Cada **usuario/proceso** debería tener **acceso únicamente a los datos y recursos necesarios** para ejecutar rutinas y tareas autorizadas
- Una cuenta de servicio no necesita ser administradora
- Una **cuenta comprometida** causa menos daño si puede hacer menos

¿Qué permisos necesita un servidor que solo publica documentos?

Principios de sistemas seguros

- Defensa multinivel
- Principio del menor privilegio (*least privilege*)
- **Separación de privilegios**

Separación de privilegios

- Tener usuarios con diferentes privilegios requiere la **división del sistema en diversas partes** (cada una con acceso limitado)
- Segmentar un sistema en componentes con necesidades de menor privilegio **previene** atacantes de **tomar el control de todo el sistema.**

Seguridad más allá del usuario

- Un **usuario** debería ser capaz de leer los ficheros que crea
- Un **proceso** NO debería leer la memoria reservada a otro proceso
- Una **App** (del móvil) NO debería editar los datos de otra app
- Un **dominio web** SOLO debería leer sus propias *cookies*
- Una **máquina confiable** debería poder acceder a un servidor de archivos

Definición de políticas de seguridad

- Sujeto (¿Quien?) actores principales (usuario, app, proceso)
- Objeto (¿Qué?) recursos (memoria, ficheros, *hardware, sockets*)
- Operación (¿Cómo?) como los sujetos operan el objeto (leer, escribir, eliminar)
  - Un usuario no debería ser capaz de borrar ficheros de otro usuario.
  - Un proceso no debería leer la memoria reservada a otro proceso

UNIX: Usuarios

- Los sistemas UNIX disponen de diferentes cuentas
  - **Cuentas de servicios**
    - Correr procesos en el *background* (p.ej. web server)
  - **Cuentas de usuarios**
    - Identificar diversos seres (¿humanos?) que interactúan con la máquina
    - Cada usuario tiene un identificador (User ID o **UID**)
      - `root` (UID 0) tiene acceso a todo
      - Algunas operaciones del sistema solo pueden ser ejecutadas por `root`

UNIX: Usuarios

```
oslab@oslab:~$ cat /etc/passwd
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/usr/sbin/nologin
man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
proxy:x:13:13:proxy:/bin:/usr/sbin/nologin
www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
backup:x:34:34:backup:/var/backups:/usr/sbin/nologin
pulse:x:128:134:PulseAudio daemon,,,:/run/pulse:/usr/sbin/nologin
gnome-initial-setup:x:129:65534::/run/gnome-initial-setup/:/bin/false
hplip:x:130:7:HPLIP system user,,,:/run/hplip:/bin/false
gdm:x:131:136:Gnome Display Manager:/var/lib/gdm3:/bin/false
oslab:x:1001:1001:,,,:/home/oslab:/bin/bash
vboxadd:x:998:1::/var/run/vboxadd:/bin/false
postgres:x:132:137:PostgreSQL administrator,,,:/var/lib/postgresql:/bin/bash
```

Nombre de la cuenta (Login)
Clave de acceso encriptada (password)
UID de esta cuenta
GID del grupo principal al que pertenece la cuenta
Nombre del usuario
Directorio de trabajo de usuario1
Interprete de comando (shell) de usuario pepito

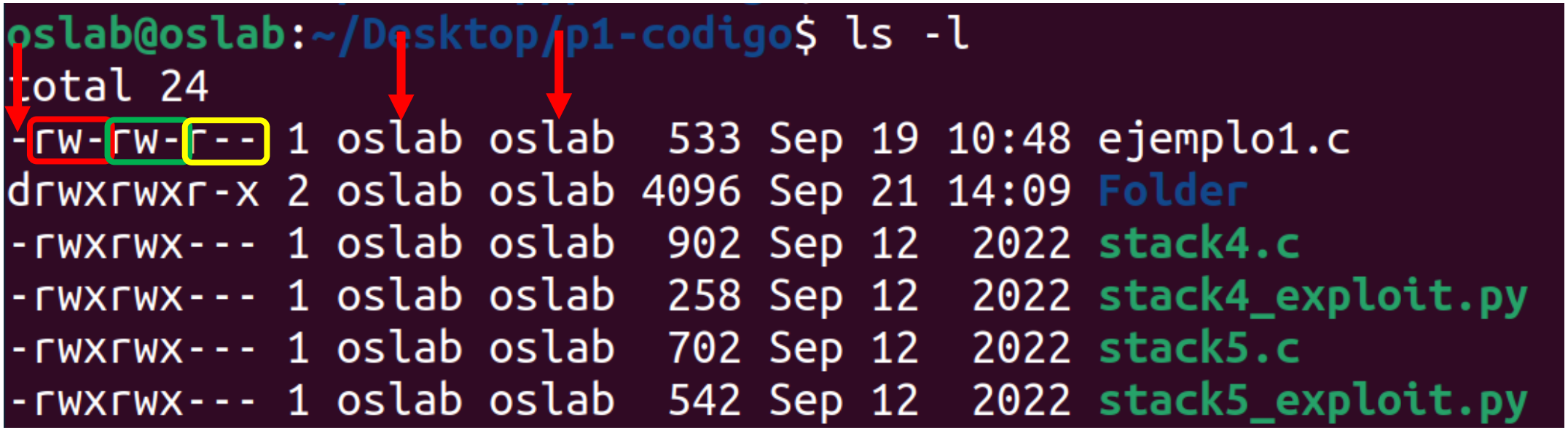
UNIX: Grupos

- UNIX tiene también grupos que utiliza para agrupar usuarios que comparten recursos (p.ej., archivo, directorio)
- Cada grupo tiene un identificador (**GID**)

```
oslab@oslab:~$ id
uid=1001(oslab) gid=1001(oslab) groups=1001(oslab),998(vboxsf),999(docker)
oslab@oslab:~$ cat /etc/group
root:x:0:
daemon:x:1:
bin:x:2:
sys:x:3:
adm:x:4:syslog,oslabadmin
```

UNIX: Propiedad de los ficheros

- Las propiedades de los ficheros y directorios muestran permisos para usuario propietario, grupo propietario, así como el resto de usuarios.



UNIX: Propiedad de los ficheros

```
directorio:   oslabadmin   grupo1   0750
fichero:      oslabadmin   grupo1   0660

seguridad1 pertenece a grupo1
```

Permiso	Valor numérico
r	4
w	2
x	1
-	0

- ¿Puede `seguridad1` leer el fichero?
- ¿Puede añadir una línea?
- ¿Puede crear otro directorio aquí?

UNIX: Access Control List

- La ***Access Control Lists*** (ACLs), pueden controlar el acceso a archivos y directorios UNIX por parte de usuarios individuales (**UID**) y grupos (**GID**)

	Desktop/	SO2/	/bin
oslab	r/w	read	read
oslabadmin	r/w	r/w	r/w

```
setfacl -option file_owner:file_permission filename
```

UNIX: creando nuevos ficheros y directorios

Mecanismo	¿Qué controla?	Idea clave
`setgid` en directorio	Grupo propietario	Los nuevos elementos heredan el grupo del directorio
`umask`	Permisos básicos `rwx`	Elimina permisos al crear ficheros/directorios
ACL por defecto	Permisos de usuarios/grupos concretos	Define permisos heredables y más granulares

Diseñar controles comprensibles

- Denegar por defecto lo que la política no autoriza
- Comprobar las operaciones que necesitan protección
- Reducir complejidad facilita revisar el mecanismo
- El diseño puede ser público y las claves seguir siendo secretas

UNIX: Procesos

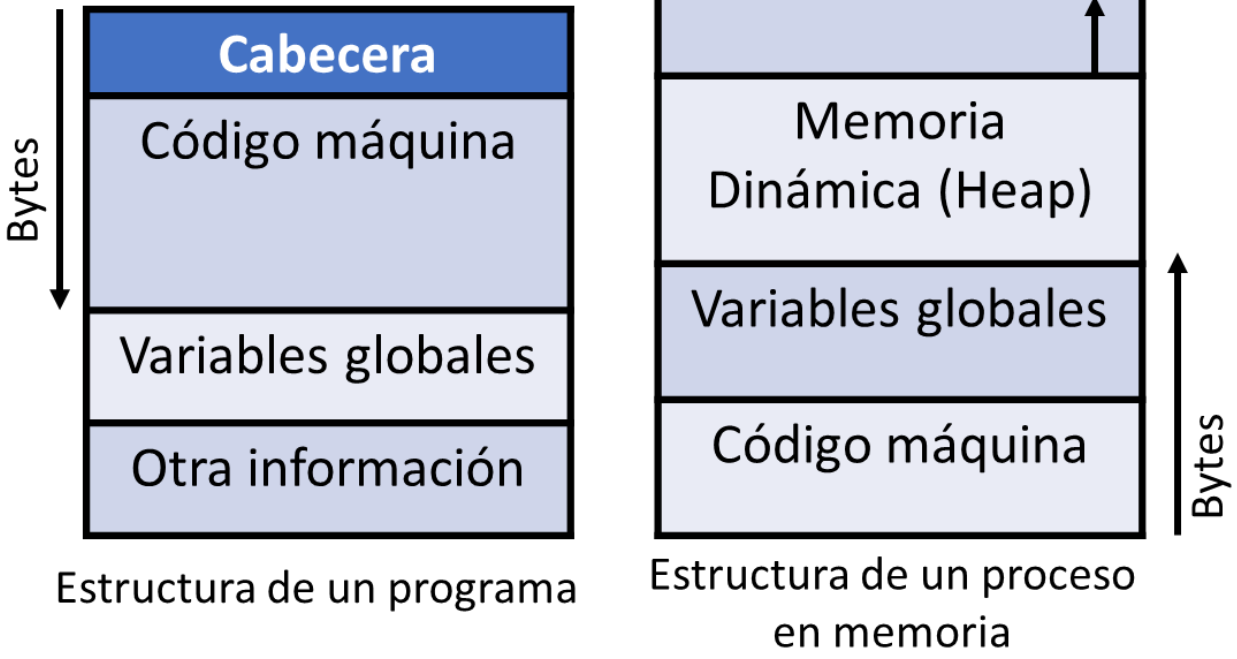
- Cada proceso tiene 3 UIDs diferentes:
  - **Effective User ID (EUID):** Determina los permisos del proceso
  - **Real User ID (RUID):** Determina el usuario que lanzó el proceso
  - **Saved User ID (SUID):** EUID antes del cambio
- `root` puede cambiar EUID, RUID y SUID
- Usuarios sin privilegios pueden cambiar EUID
- `setuid(x)` permite a un proceso cambiar temporalmente su EUID por otro

UNIX: Procesos

- Muchos procesos comienzan como `root` y hacen `fork`

```
oslab@oslab:~/Desktop/p1-codigo$ pstree -u
systemd─┬─ModemManager───2*[{ModemManager}]
        ├─NetworkManager───2*[{NetworkManager}]
        ├─VBoxDRMClient───4*[{VBoxDRMClient}]
        ├─VBoxService───8*[{VBoxService}]
        ├─accounts-daemon───2*[{accounts-daemon}]
        ├─acpid
        ├─avahi-daemon(avahi)───avahi-daemon
        ├─colord(colord)───2*[{colord}]
        ├─containerd───7*[{containerd}]
        ├─cron
        ├─cups-browsed───2*[{cups-browsed}]
        ├─cupsd
        ├─dbus-daemon(messagebus)
        ├─dockerd───8*[{dockerd}]
        ├─fprintd───4*[{fprintd}]
        ├─gdm3─┬─gdm-session-wor─┬─gdm-wayland-ses(oslab)─┬─gnome-session-b───2*[{gnome-session-b}]
        │      │                 │                        └─2*[{gdm-wayland-ses}]
        │      │                 └─3*[{gdm-session-wor}]
        │      └─2*[{gdm3}]
        ├─gnome-keyring-d(oslab)───3*[{gnome-keyring-d}]
```

Buffer overflow





Buffer overflow

```
#include<stdio.h>
void get_name(){
   char buffer[10];
   gets(buffer);
}
int main() {
     printf("Please enter your name.");
     get_name();
     return 0;
}
```

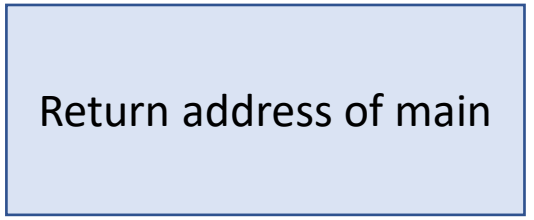
Call stack



Buffer overflow

```
#include<stdio.h>
void get_name(){
   char buffer[10];
   gets(buffer);
}
int main() {
    printf("Please enter your name.");
    get_name();
    return 0;
}
```

Call stack



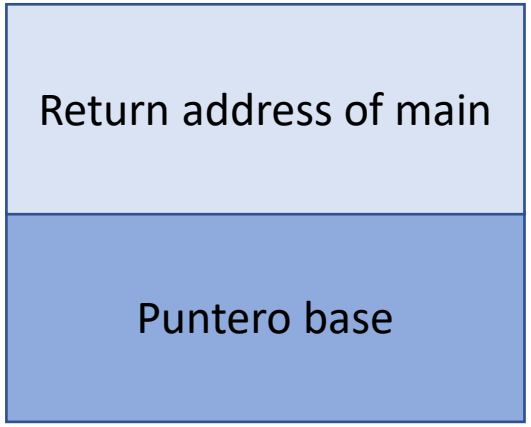
`main()` llama a `get_name()`

La dirección de retorno de `main` se inserta en la pila.

Buffer overflow

```
#include<stdio.h>
void get_name(){
   char buffer[10];
   gets(buffer);
}
int main() {
    printf("Please enter your name.");
    get_name();
    return 0;
}
```

Call stack

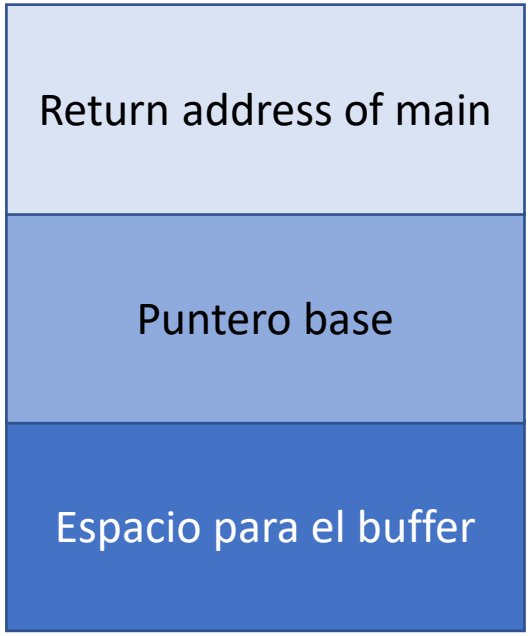


El puntero base se inserta en la pila

Buffer overflow

```
#include<stdio.h>
void get_name(){
   char buffer[10];
   gets(buffer);
}
int main() {
     printf("Please enter your name.");
     get_name();
     return 0;
}
```

Call stack

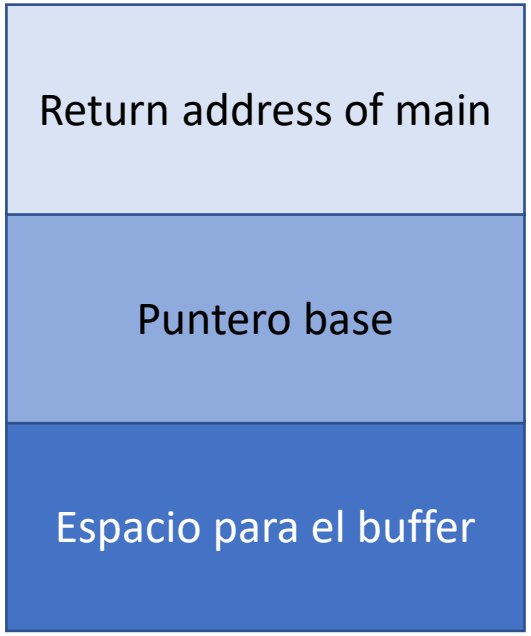


Espacio para las variables locales de `get_name()`. Se asigna espacio específico al buffer.

Buffer overflow

```
#include<stdio.h>
void get_name(){
   char buffer[10];
   gets(buffer);
}
int main() {
    printf("Please enter your name.");
    get_name();
    return 0;
}
```

Call stack



Si se inserta 10+ caracteres, la capacidad del búfer excederá → desbordamiento del búfer. Podría sobrescribir el puntero base y la dirección de retorno del `main`.

¿Qué problemes puede ocasionar?

- Explotación de vulnerabilidades
  - Corrupción de Datos
  - Ejecución de código malicioso
  - Elevación de privilegios
  - Denegación de Servicios (DoS)
  - Fuga de Información Confidencial
  - Compromiso del sistema
  - Perdida de confianza

Tipos de buffer overflow: stack overflow

- Un ***stack overflow*** (desbordamiento de pila) es un tipo de error de programación que ocurre cuando una aplicación o programa supera el límite de la memoria reservada para su pila de llamadas (*stack*)



Tipos de buffer overflow: heap overflow

- Un ***heap overflow*** (desbordamiento de la memoria dinámica) es una vulnerabilidad de seguridad en la que un programa sobrescribe accidental o maliciosamente datos en la región de la memoria conocida como (*heap*) más allá de los límites de un buffer o estructura de datos asignada.

Memoria de datos vs memoria ejecutable

- Código y datos son lo mismo físicamente (*bits*), pero se tratan distinto a nivel de permisos
- Si una página de la pila está marcada como no ejecutable, CPU impedirá que se interpreten instrucciones aunque contenga *bytes* que formen código válido. Evitando así que un atacante simplemente inyecte *bytes* en la pila y los ejecute (inyección directa)
- Ver `mprotect`, `mmap` y `cat /proc/<PID>/maps` o `pmap <PID>`

Mecanismos para evitar un buffer overflow

- Canarios (*Canary*)
- Address Space Layout Randomization (ASLR)
- Data Execution Prevention (DEP)
- Position Independent Executable(PIE)
- Limitación de la longitud de entrada
- Uso de funciones seguras
- Actualizaciones y parches

- gets() → fgets() // lee caracteres
- strcpy() → strncpy() // copia el contenido del buffer
- strcat() → strncat() // concatenación de buffer
- sprintf() → snprintf() // llena el buffer con datos de diferentes tipos

Canario

- El **compilador** puede tomar medidas de protección para protegerse contra el *buffer overflow*.
- Uno de estos mecanismos es un **canario (Canary)**







Canario

- Un canario es un **valor generado aleatoriamente** y, en el caso de un desbordamiento del búfer, el canario se sobrescribe. Al compararlo con el valor conocido, el compilador detecta que la pila estaba comprometida y devuelve un `stack smashing detected error.`

Call stack





```
gcc overflow.c -o overflow -fno-stack-protector
```

Canario





ASLR

- ASLR (*Address Space Layout Randomization*) es una técnica de seguridad que **aleatoriza la ubicación de las áreas de memoria importantes** (pila, el *heap* y las bibliotecas compartidas) en el espacio de direcciones del proceso.
- Hace que sea mucho más difícil para un atacante predecir dónde se encuentra el código, las estructuras de datos y otros elementos críticos.

DEP

- DEP (*Data Execution Prevention*) es una característica de seguridad que **previene la ejecución de código en regiones de memoria que deberían contener solo datos**, como la pila o *heap*. DEP marca áreas de memoria como no ejecutables.
- DEP dificulta la ejecución de **código inyectado** por un atacante en áreas de memoria destinadas solo para datos, reduciendo la efectividad de los ataques de desbordamiento de búfer.

PIE

- PIE (*Position Independent Executable*) es una medida de seguridad que hace que **el ejecutable pueda cargarse en memoria en direcciones distintas en cada ejecución**.
- Permite que ASLR aleatorice también la base del propio binario, no solo las librerías o la pila.

La semana que viene no hay session Teo ni Lab

Crear grupos de laboratorios

Jueves 1-oct test tema Seguridad SOs

